Project Work

Message Classification:
Spam or Ham

Alessia Lettieri
0622702324

Marco Panico
0622702416

Topics

01 Machine Learning

Training machine learning models accurate predictions for Message Classification (Ham or Spam) .

02 Vulnerabilities

Application vulnerabilities that can be exploited to compromise its security.

03 Architecture

Workload Distribution
Dinamic Scalability

04 Kubernetes

Deployment, scaling, and management of containerized application and stress test.

05 Security

Strategies for ensuring application security.

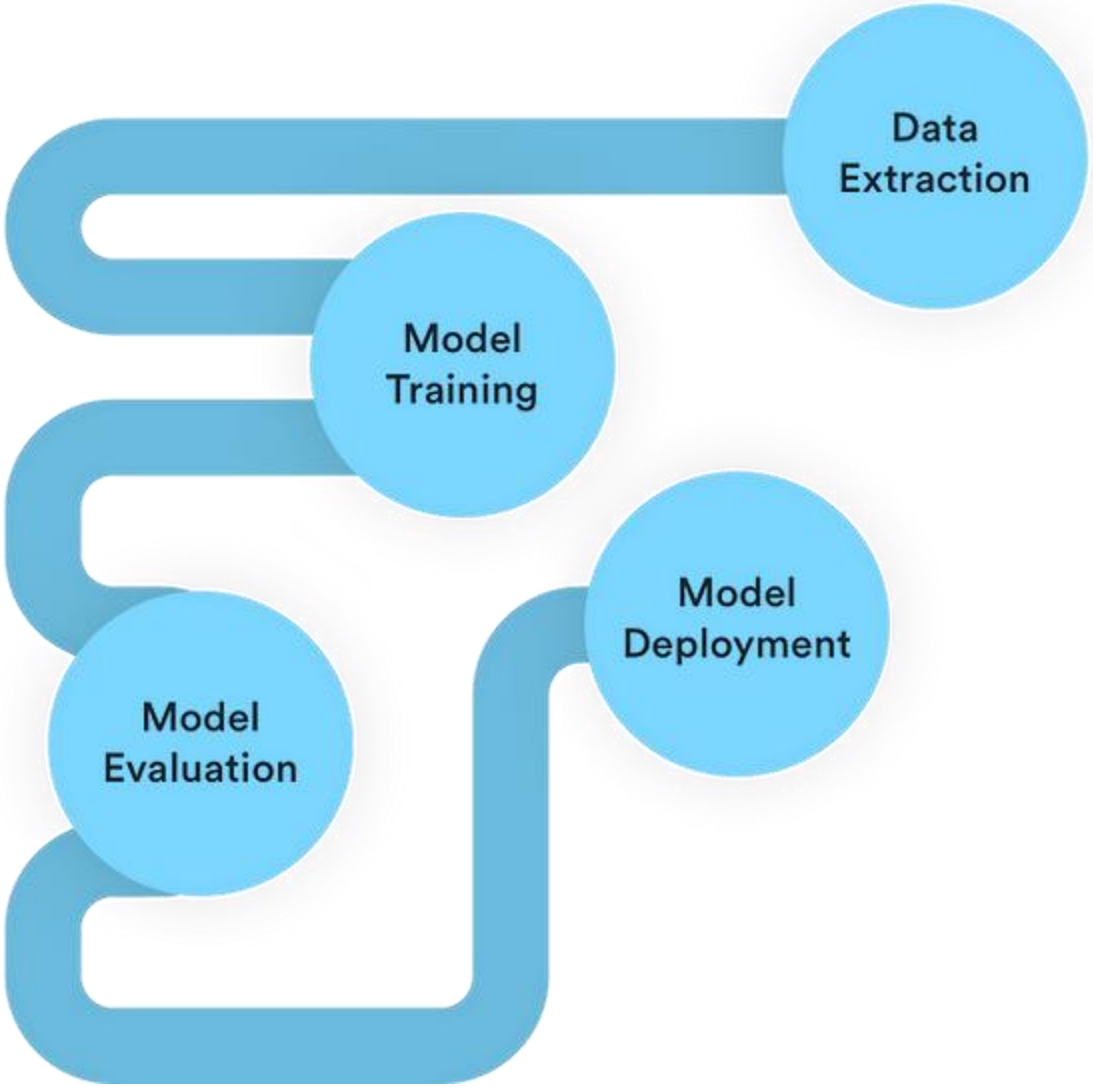
Natural Language Preprocessing

NLP is a branch of Computer Science where we study about the processing of Text Data and Human Language.

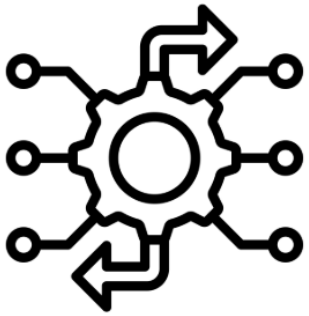
Traditionally, computers are only able to process numerical data.

We map the given text input or language data into the numerical representation of it thereafter we apply machine learning algorithms

Machine Learning Models

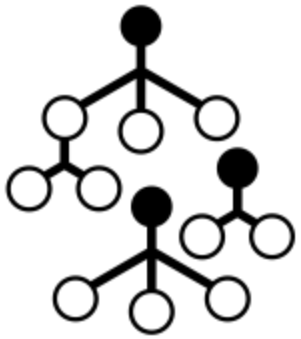


Pipeline



TfidfVectorizer

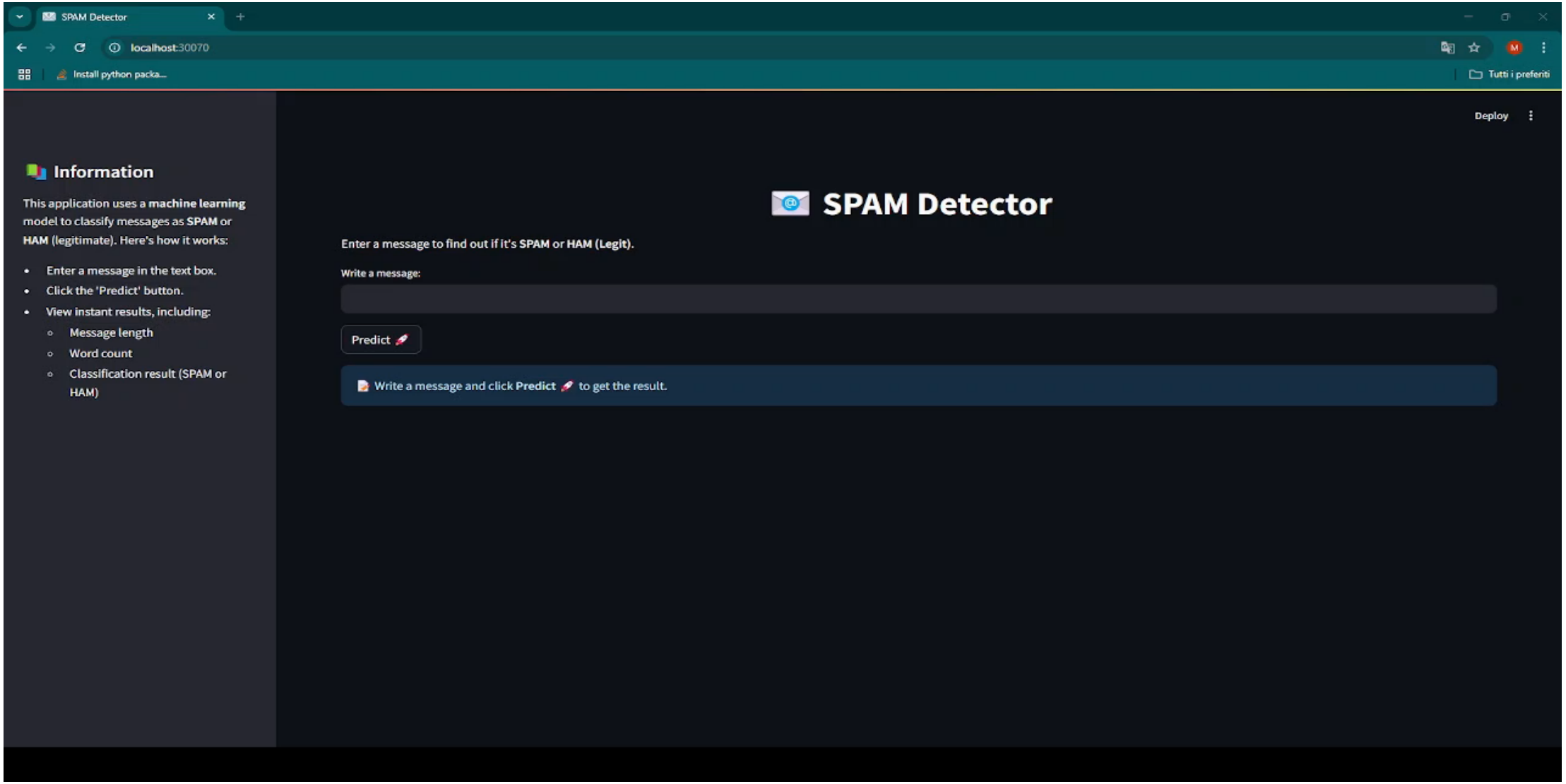
Transforming text into numeric representation



Random Forest Classifier

Ability to handle complex relationships within the data and capture non-linear patterns.

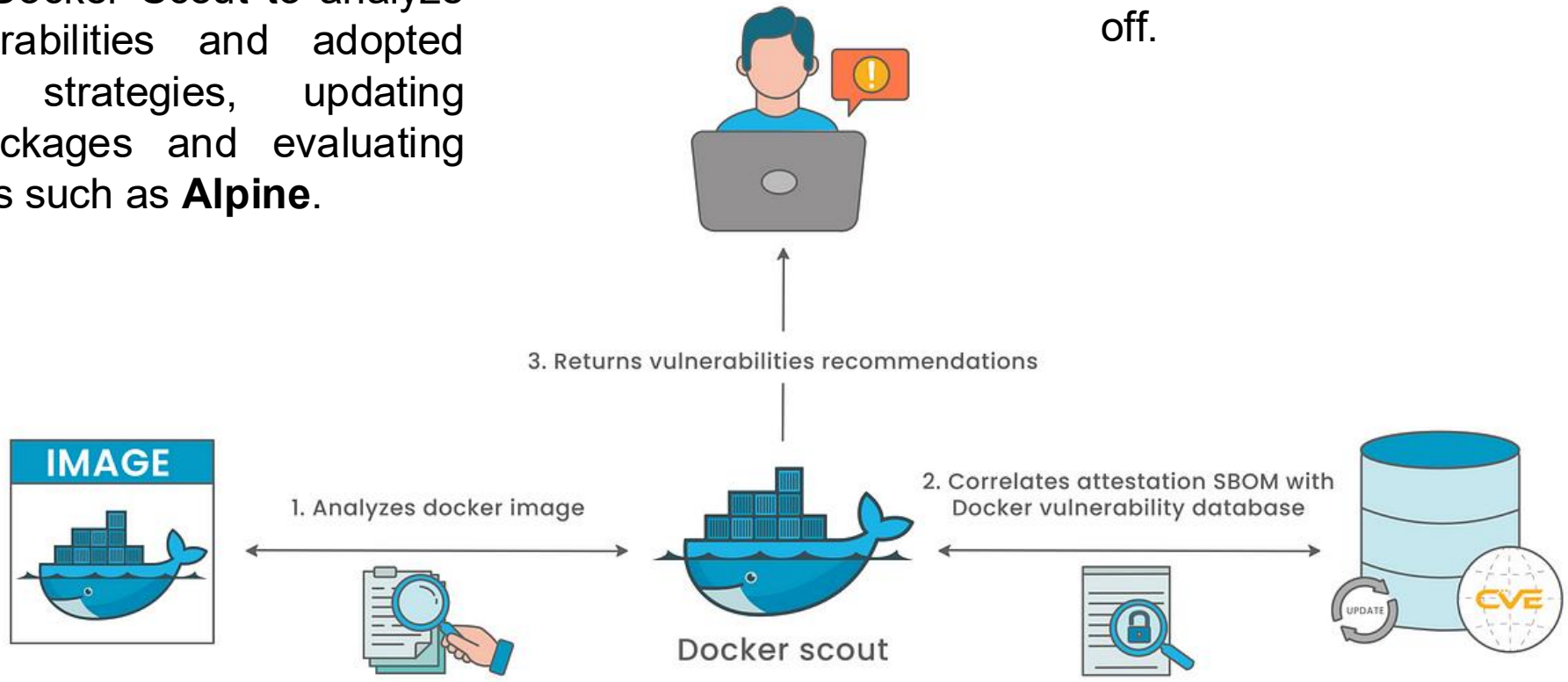
Web App


SPAM Detector
localhost:30070
Install python packa...
Tutti i preferiti
Deploy
Information
This application uses a machine learning model to classify messages as SPAM or HAM (legitimate). Here's how it works:
Enter a message in the text box.
Click the 'Predict' button.
View instant results, including:
Message length
Word count
Classification result (SPAM or HAM)
SPAM Detector
Enter a message to find out if it's SPAM or HAM (Legit).
Write a message:
Predict
Write a message and click Predict to get the result.

Vulnerabilities

The vulnerability analysis revealed some problems with the use of pre-existing Docker images, **python:3.10-slim-buster**, which introduced some security issues. We used Docker Scout to analyze the vulnerabilities and adopted mitigation strategies, updating critical packages and evaluating alternatives such as **Alpine**.

However, to ensure compatibility and stability, we decided to maintain the python image, accepting some security trade-off.



Python image Vulnerabilities Fixed :

- CVE-2024-6345 in **setuptools**
  Allows Remote Code Execution (RCE).
  Mitigation: Update to setuptools >= 70.0.
- CVE-2023-5752 in **pip**
  Mitigation: Update to pip >= 23.3.1

Unfixed Vulnerabilities

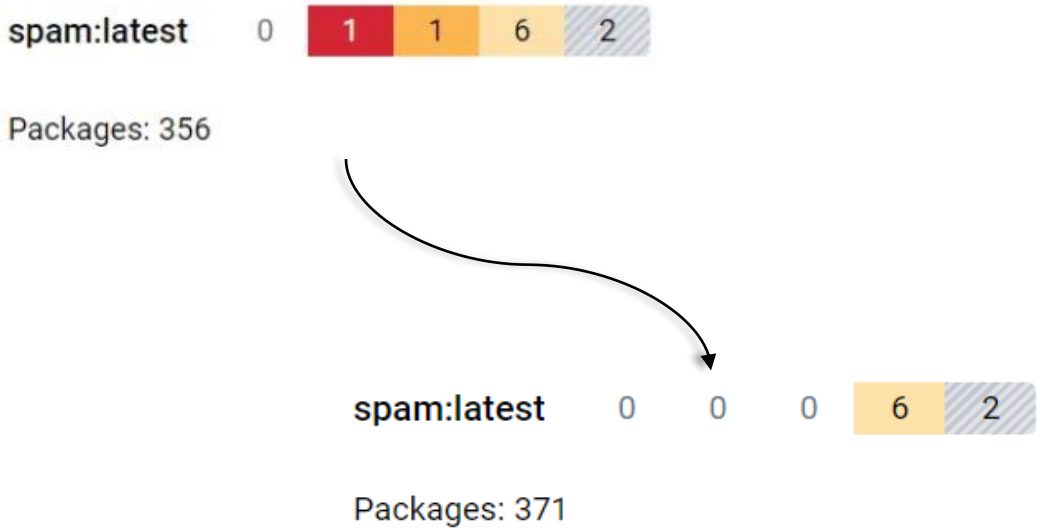
- CVE-2010-0928: **OpenSSL** issue.
- CVE-2023-36191: **SQLite3** v3.40.1 issue.

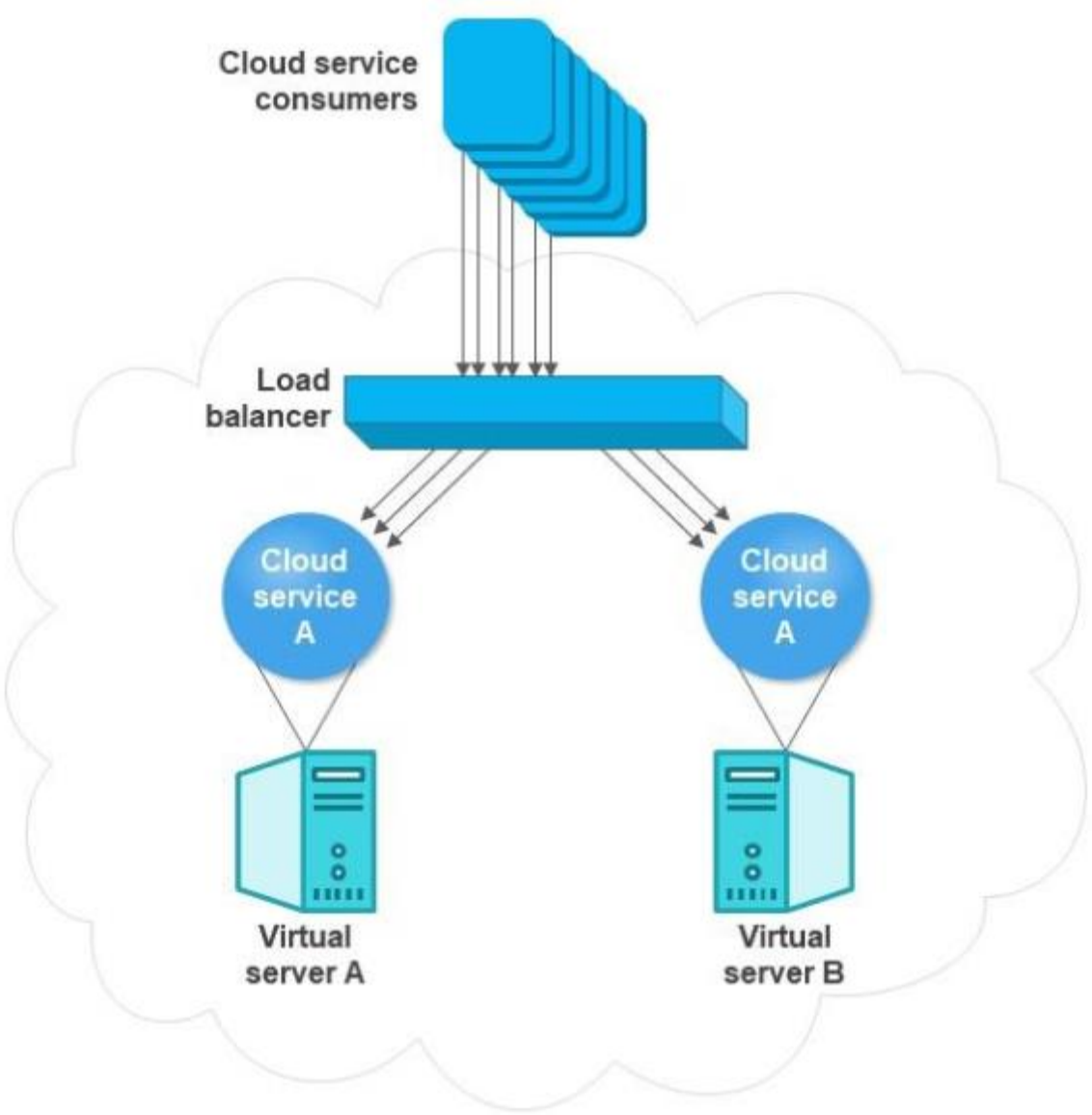
The ***spam:latest image*** has eight unfixed vulnerabilities:

- Two vulnerabilities inherited from Python image.
- Four vulnerabilities in the **debian/binutils**
- Two vulnerabilities in the **debian/glib**

Legend Severity

- High
- Medium
- Low
- Unspecified





Workload Distribution Architecture

Composition:

- Cloud service consumers send requests to the **load balancer**.
- The load balancer distributes the requests across multiple **cloud services** A
- Each cloud service A is managed by **virtual servers** (A and B).

Motivation

01 Load Characteristics:

String analysis is less intensive than complex data or images.

No large computing or **storage resources** are required, making Workload Distribution Architecture particularly suitable.

02 Uniform distribution of the workload:

Load balancing evenly distributes the workload across multiple IT resources

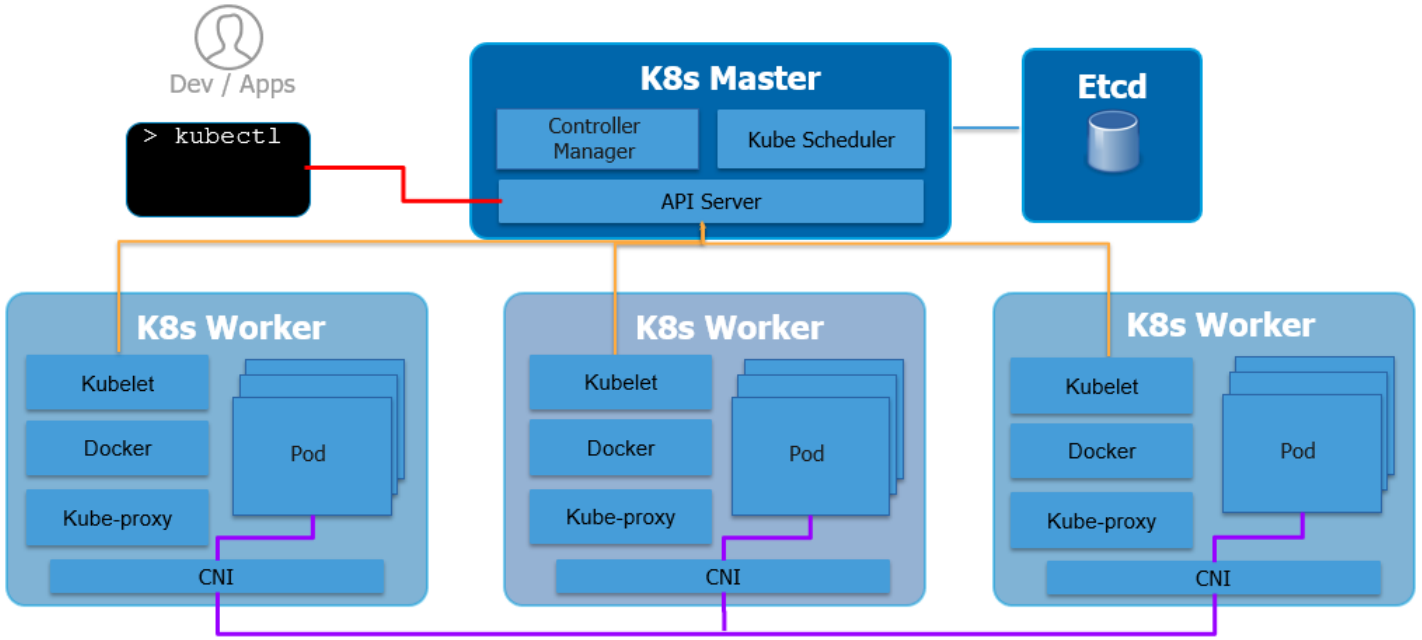
Reducing the risk of overload and ensuring more efficient management of resources.

03 Redundancy & Reliability

Traffic is **automatically redirected** in the event of a fault, avoiding service interruptions.

It **guarantees continuous** service even for a large number of consumers.

Kubernetes Architecture



1 Control Node

Control center, coordinating tasks in Kubernetes cluster

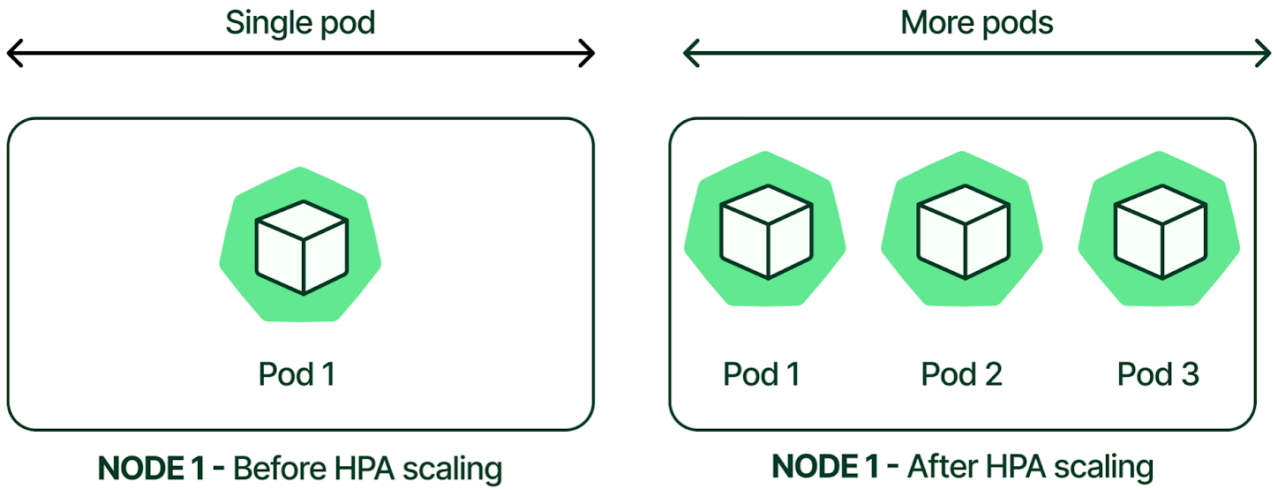
2 Worker Node

To improve capacity, load distribution and availability

Replicas

Improves application availability and resilience

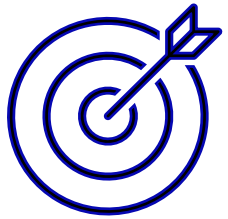
Horizontal Pod Autoscaler



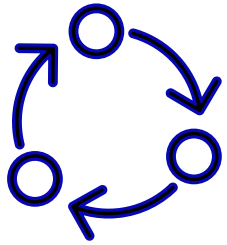
01 Ensuring scalabity and responsiveness.

02 Handling HTTP fluctuation.

Pod Size



Determine the right initial CPU and memory size for the main pod by making a measurement of the overhead introduced by Kubernetes and the container runtime.



1. Stress testing with continuous monitoring via Kubernetes Metrics Server
2. Comparison of main pod and whole node consumption

Resource	Node Usage	Pod Usage	Estimated Overhead
CPU	987m	946m	41m
Memory	260Mi	47Mi	213Mi

01 CPU

String processing and model inference are CPU-intensive, requiring more compute than memory.

There is no significant difference between pod and node CPU usage → most of the CPU is used directly by the app.

02 MEMORY

The pod uses minimal memory

The node's higher memory usage (260Mi) is due to system components (kubelet, Docker runtime, Other background services)

Manual Scaling

Objective

Identify the optimal number of pod replicas to:

- Handle **5,000 simultaneous requests**
- Support a daily load of **50,000 users**
- Ensure **zero errors** and **low avarange latency** (one second)

Manual Scaling

Objective

Identify the optimal number of pod replicas to:

- Handle **5,000 simultaneous requests**
- Support a daily load of **50,000 users**
- Ensure **zero errors** and **low avarange latency** (one second)

Why Manual Scaling?

- **Controlled** environment for testing
- Step-by-step **performance evaluation**
- Establish a **baseline** before applying **autoscaling (HPA)**

Manual Scaling

Objective

Identify the optimal number of pod replicas to:

- Handle **5,000 simultaneous requests**
- Support a daily load of **50,000 users**
- Ensure **zero errors** and **low avarange latency** (one second)

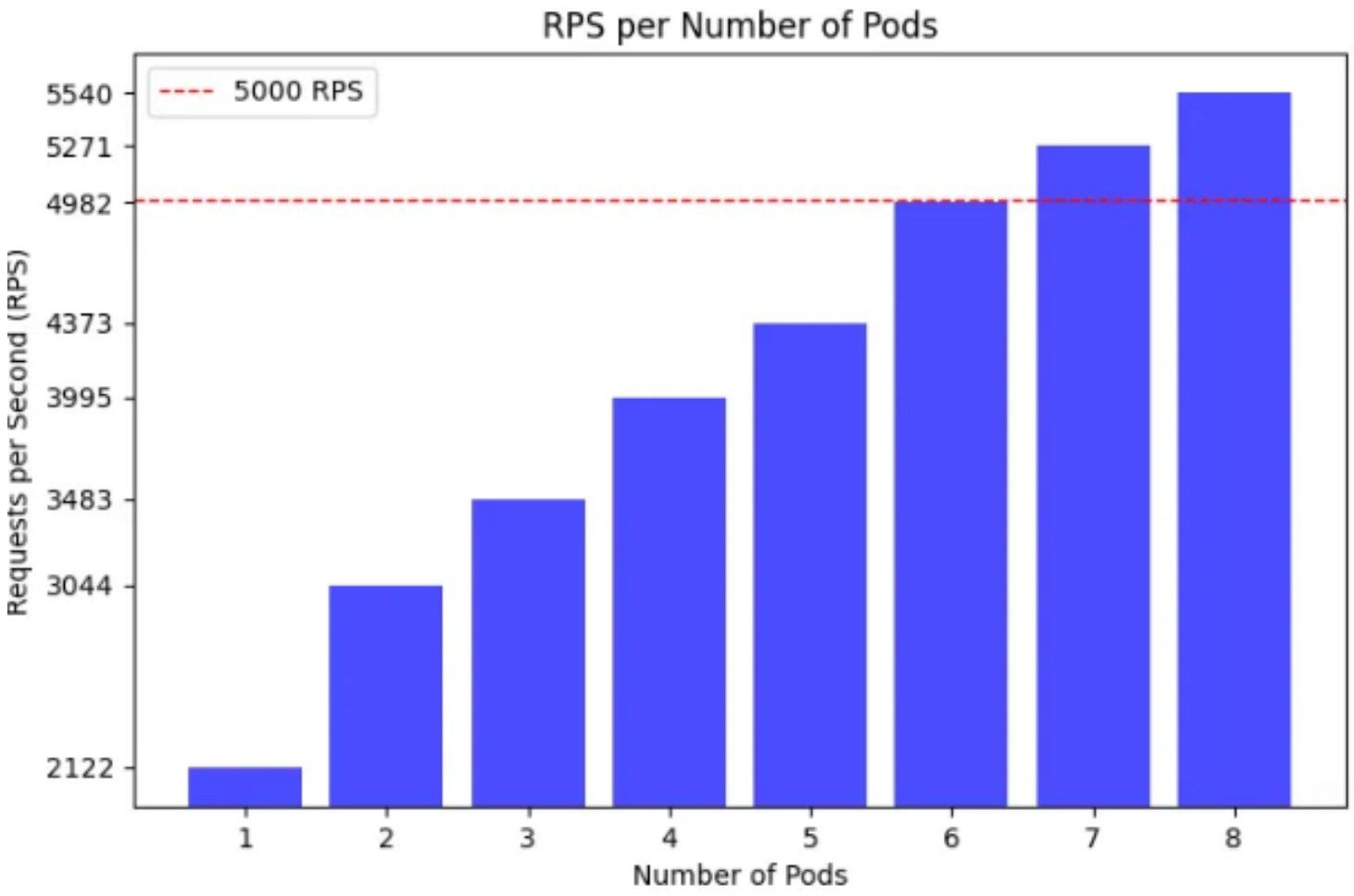
Why Manual Scaling?

- **Controlled** environment for testing
- Step-by-step **performance evaluation**
- Establish a **baseline** before applying **autoscaling (HPA)**

Testing Approach

Tests executed incrementally from **1 to 8 replicas**

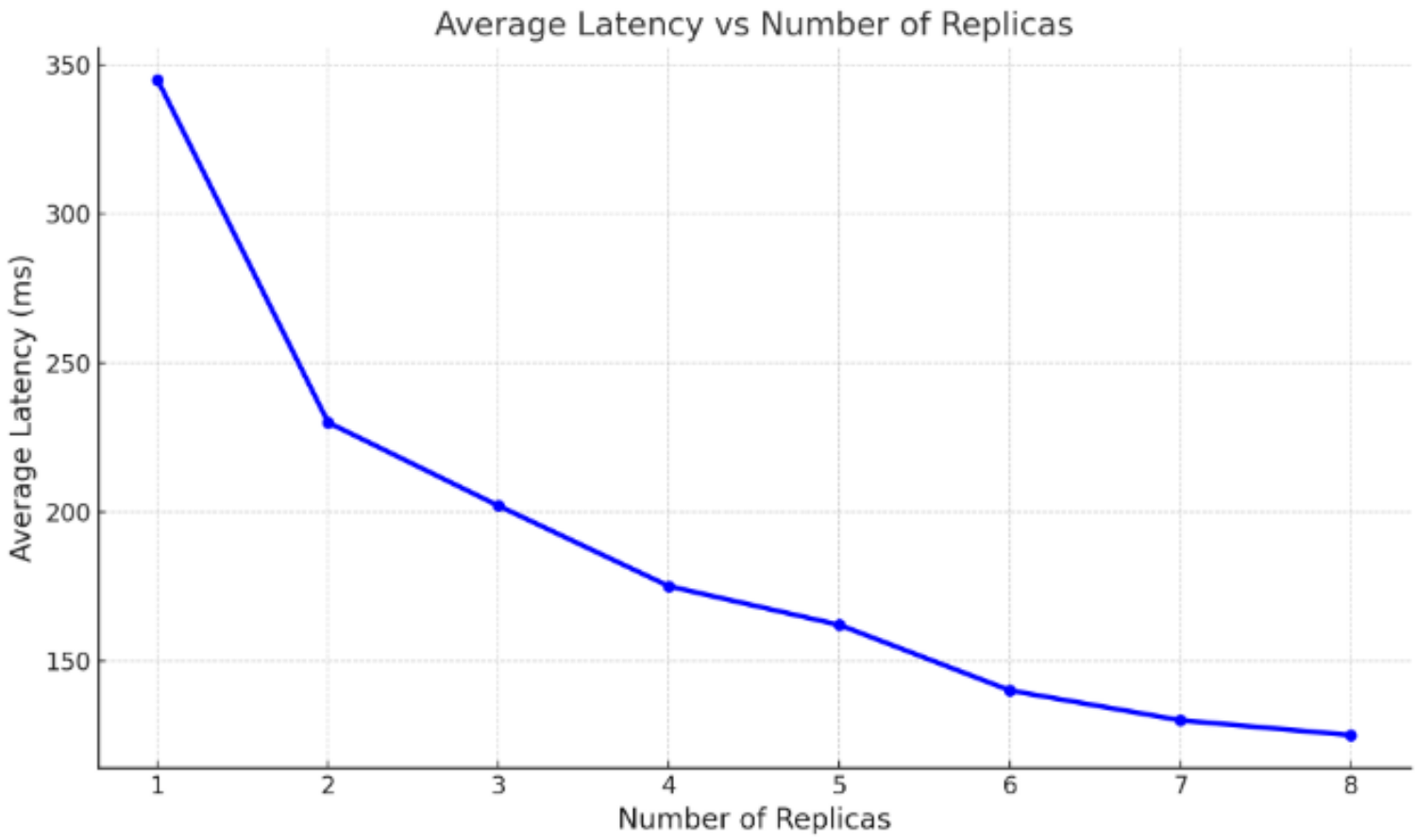
Request per second



Increasing the number of pods leads to a progressive increase in Requests per Second (RPS).

- With **6 pods**, the system approaches the target of 5000 RPS, but doesn't clearly exceed it.
- **7 pods** bring the system just above the required threshold.
- With **8 pods**, a safety margin is achieved, useful for handling potential traffic spikes, ensuring reliability and service continuity.

Average latency



The average latency decreases as the number of replicas increases.

Significant improvements are observed up to 6 replicas.

7 and 8 replicas provide slight improvements in response time and ensure greater stability.

More replicas → lower latency and a more responsive system

Horizontal Pod Autoscaler

Objective

Identify the optimal number of pod replicas to:

- Handle **5,000 simultaneous requests**
- Support a daily load of **50,000 users**
- Ensure **zero errors** and **low avarange latency** (one second)

Why Scale Horizontally?

- Adaptability to **variable workloads**
- **Resource optimization**
- Improved **resilience** and **responsiveness**
- The **stateless nature** of the application allows independent request handling

Horizontal Pod Autoscaler

Objective

Identify the optimal number of pod replicas to:

- Handle **5,000 simultaneous requests**
- Support a daily load of **50,000 users**
- Ensure **zero errors** and **low avarange latency** (one second)

Why Scale Horizontally?

- Adaptability to **variable workloads**
- **Resource optimization**
- Improved **resilience** and **responsiveness**
- The **stateless nature** of the application allows independent request handling

Testing Approach

Scales based on **CPU usage**

Evaluates system behavior under **simulated load** across different **scaling configurations**

Stress Test: Configuration 2-8

```
Result:
    total: 600233 requests
    errors: 0 errors
    error percentage: 0.0%
    latency min: 7.1889ms
    latency median: 142.662ms
    latency average: 139.824592ms
    latency max: 1.2257883s
    transfers: 1102628021 bytes
    rps: 5006.3 requests per sec
```

Start with 2 Active Pods.

More **efficient** under low traffic conditions.

Slower response to sudden traffic spikes, as it **takes longer to scale up** to the required number of pods.

Capable of handling up to 5,000 simultaneous requests, **but with more gradual adaptation**

Stress Test: Configuration 3-8

```
Result:
    total: 641068 requests
    errors: 0 errors
    error percentage: 0.0%
    latency min: 2.3176ms
    latency median: 128.03365ms
    latency average: 130.935813ms
    latency max: 1.1766784s
    transfers: 1177641916 bytes
    rps: 5346.1 requests per sec
```

Start with 3 Active Pods.

Higher **initial capacity** available.

Uses **more resources** when traffic is low.

Better **readiness** and **responsiveness** to traffic spikes.

Preferred for greater resilience, stable latency, and the ability to absorb sudden traffic spikes

Results Comparison

HPA Configuration	RPS	Average Latency	Initial Replicas	Max Replicas
Manual	5,540	128 ms	8	8
HPA	5,346	130.9 ms	3	8
HPA	5,006	139.8 ms	2	8

01 Manual

- Best latency & RPS overall
- Ideal for constant high load

02 HPA 3-8

- Near-equal performance to manual
- Fast spike handling, good balance

03 HPA 2-8

- Most efficient at idle
- Slower under sudden peaks – better for gradual loads

Results Comparison

HPA Configuration	RPS	Average Latency	Initial Replicas	Max Replicas
Manual	5,540	128 ms	8	8
HPA	5,346	130.9 ms	3	8
HPA	5,006	139.8 ms	2	8

01 Manual

- Best latency & RPS overall
- Ideal for constant high load

02 HPA 3-8

- Near-equal performance to manual
- Fast spike handling, good balance

FINAL CHOICE

03 HPA 2-8

- Most efficient at idle
- Slower under sudden peaks – better for gradual loads

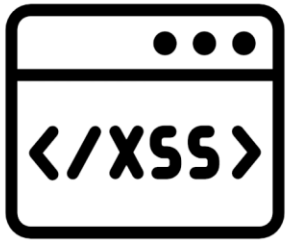
Application Vulnerabilities

01 SQL injection

Allows malicious queries to be executed on the database, compromising its confidentiality or integrity

02 Cross-Site Scripting (XSS)

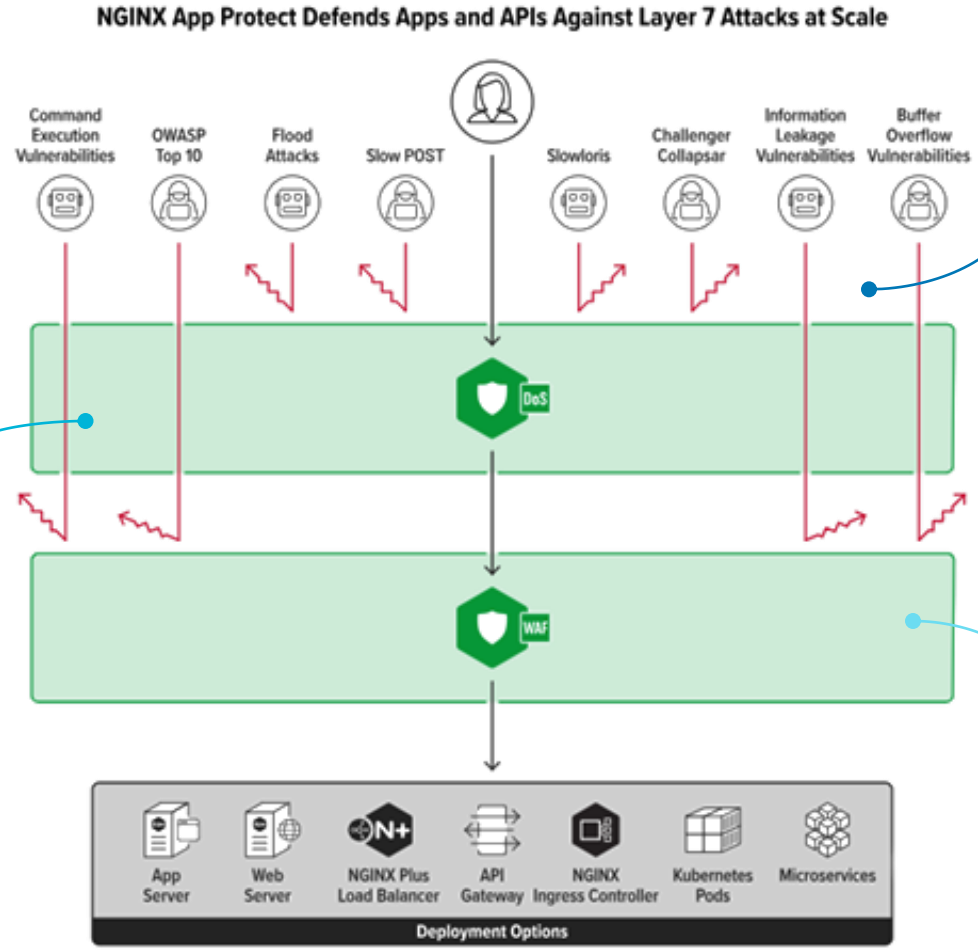
Allow an attacker to inject malicious scripts into web pages, putting user data at risk.



03 DDoS

These attacks are characterized by a massive volume of traffic generated by a distributed network of compromised devices

Protecting Apps from Hacks



Vulnerabilities

The web app may face SQL injection, XSS and software vulnerabilities.

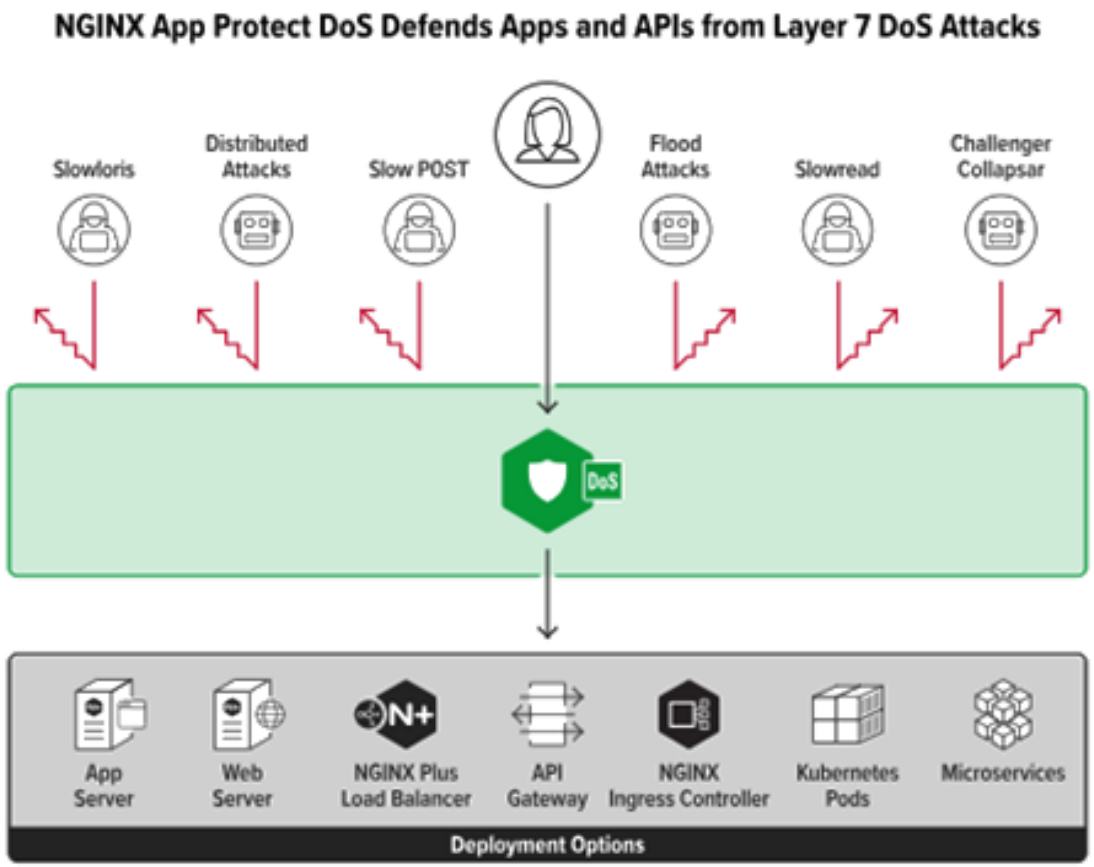
App Protect DoS

Create rules for spotting bad traffic in a baseline model.

App Protect WAF

Detects XSS, SQL Injections, and other app layer attacks.

Protection with NGINX and App Protect DoS



01 Simplifies the setup, speeds up the time it takes to launch applications.

02 Improves resilience, provides better visibility, and enhances security.

03 Offers advantages like modular and reusable components.

04 Handles DoS attacks.

THANK YOU!

Any questions?